16.3 Dealing with exceptions

The supervision model in Akka Streams is the same as you learned about in previous chapters for basic behaviors, but its notation is a bit different. When you deal with an exception in a stream, you can use `recover` to set the supervision strategy. However, with `recover`, you only get the last element before the stream stops. This is similar to an actor that stops by default once it gets an exception.

But before you add recovery, you should `log` all risky operations in your stream. If you don't add logging, you won't know whether an exception was thrown. Without logging, the stream will be stopped, but you won't get any information about it. The following listing adds a log to a source that fails often.

Listing 16.5 Exception caused by an element

```
class ExceptionsSpec ... {
  ...
  "an Exception" should {
    "stop the stream and log 'tried riskyHandle'" in {

      def riskyHandler(elem: Int): Int =     ❶
        100 / elem

      val result =
        Source(-1 to 1)
          .map(riskyHandler)
          .log("tried riskyHandler")         ❷
          .map(println)
          .run()

      Await.result(result, 1.seconds)
    }
  }
  ...
}
```

❶ Handler that fails at elem = 0